

DATABASE_SCHEMA

Helix Cluster OS — Database Schema Reference

PostgreSQL 16+ is the authoritative relational store for the Helix Cluster control plane: the node registry, GPU inventory, sessions and their window/pane tree, resource reservations, the scheduling queue, health snapshots, LLM advisories, the build subsystem, users, the migration-history ledger, network policy, cluster config, and an append-only audit log.

This document is generated by reading the real SQL under `migrations/postgresql/`. Every table, column, constraint, index, trigger, and function below is grounded in those files. Where two sources disagree, that divergence is called out explicitly rather than papered over.

1. Two schema sources — read this first

There are **two distinct, independently-applied** schema definitions in `migrations/postgresql/`, and they are **not byte-identical**. Understanding which one your environment runs is essential before trusting any column/constraint detail.

1a. The `golang-migrate` chain (001–015, what `make migrate-up` applies)

The numbered pairs `001_*.up.sql ... 015_*.up.sql` (with matching `*.down.sql`) are the `golang-migrate` migration chain. This is the set applied by `make migrate-up`:

- Makefile target `migrate-up` runs `bash scripts/run-migrations.sh up` (Makefile:78–80).
- `scripts/run-migrations.sh` invokes the `migrate` binary against `migrations/postgresql/` (`scripts/run-migrations.sh:59–67`).
- `golang-migrate` only recognizes files matching `{version}_{title}.{up|down}.sql`. The numbered pairs match; `0001_primary_schema.sql` does **not** (it has no `.up.sql/.down.sql` suffix), so the migrate runner **ignores** it.

This chain creates 14 tables (no CHECK constraints, fewer indexes, and some **different table names** — see §1c) plus the trigger/function set in migration 015.

1b. 0001_primary_schema.sql (the consolidated “authoritative” schema)

0001_primary_schema.sql is a single consolidated file (header: “*Authoritative Postgres 16+ Primary Schema — 15 tables, CHECK constraints, ~53 indexes, triggers, audit*”). It is **not** part of the golang-migrate chain (see §1a). Instead it is applied directly by Go code:

- internal/schema/schema.go SchemaPath() resolves migrations/postgresql/0001_primary_schema.sql (internal/schema/schema.go:108).
- ApplyPrimarySchema() streams the file to psql -f - over stdin, for both local-DSN and podman exec (container) transports (internal/schema/schema.go:279–308).

This file defines **15 tables** with full CHECK constraints, GIN indexes, helix_-prefixed triggers/functions, an audit_log DEFAULT partition, partitions through 2027-06, and seed rows for cluster_config.

Honesty note (CLAUDE-1/CLAUDE-3): the project ships *both* a migrate chain and a consolidated schema, and they differ. The full distributed/production application path (which source is canonical in a deployed cluster, and reconciliation between the two) is **not validated end-to-end here** — this reference documents what each SQL file declares, not a single unified deployed database. Treat 0001_primary_schema.sql as the richer, constraint-complete definition and the 001–015 chain as the migrate-driven baseline.

1c. Where the two sources diverge

Concept	golang-migrate chain (001–015)	0001_primary_schema.sql
Reservations table	resource_allocations (006)	reservations
Health snapshots table	health_scores (008)	health_snapshots
Advisories table	advisories (009)	llm_advisories
Scheduling queue	scheduling_queue (007) — present	absent
network_policies	absent	present (table 14)
cluster_config	absent	present (table 15, with seed rows)
CHECK constraints	none	extensive (status enums, ranges)
audit_log partitions	2026-06 ... 2026-12 (7)	+ 2027-01 ... 2027-06, + DEFAULT
audit_log severity CHECK	none	DEBUG/INFO/WARNING/ERROR/CRITICAL
Trigger/function names	update_updated_at_column, audit_trigger	helix_update_updated_at, helix_audit_trigger
build_artifacts.artifact_type	absent	present (BINARY/CONTAINER_IMAGE/...)
	nullable (DEFAULT only)	NOT NULL DEFAULT

Concept	golang-migrate chain (001-015)	0001_primary_schema.sql
Default-value NOT NULL on labels/ features/details/environment/ metrics		

The per-table sections below document the **0001_primary_schema.sql** definitions as the primary reference (it is the superset with constraints), and note the migrate-chain differences inline.

2. Ordered migration list (golang-migrate chain)

Applied in order by make migrate-up. Each *.up.sql has a paired *.down.sql that drops the indexes and table(s) it created.

#	File	Adds
001	001_create_nodes.up.sql	nodes table + 4 indexes (status, role, region, GIN labels)
002	002_create_gpu_devices.up.sql	gpu_devices table (FK → nodes ON DELETE CASCADE) + 3 indexes
003	003_create_sessions.up.sql	sessions table (FK → nodes) + 4 indexes
004	004_create_session_windows.up.sql	session_windows table (FK → sessions CASCADE) + 2 indexes
005	005_create_session_panes.up.sql	session_panes table (FKs → session_windows CASCADE, nodes, gpu_devices) + 4 indexes
006	006_create_resource_allocations.up.sql	resource_allocations table (FKs → sessions, nodes) + 4 indexes
007	007_create_scheduling_queue.up.sql	scheduling_queue table + 3 indexes
008	008_create_health_scores.up.sql	health_scores table (FK → nodes) + 3 indexes
009	009_create_advisories.up.sql	advisories table + 3 indexes
010	010_create_audit_log.up.sql	audit_log partitioned table + 7 monthly partitions (2026-06... 2026-12) + 4 indexes
011	011_create_build_jobs.up.sql	build_jobs table (FK → nodes) + 4 indexes
012	012_create_build_artifacts.up.sql	build_artifacts table (FK → build_jobs CASCADE) + 3 indexes

#	File	Adds
013	013_create_users.up.sql	users table (UNIQUE spiffe_id) + 1 index
014	014_create_migration_history.up.sql	migration_history table (FKs → sessions, nodes×2) + 3 indexes
015	015_triggers_and_functions.up.sql	update_updated_at_column() + audit_trigger() functions; updated_at triggers on 7 tables; audit triggers on nodes/sessions/build_jobs/users

The consolidated 0001_primary_schema.sql declares the same families plus reservations, health_snapshots, llm_advisories, network_policies, cluster_config and all CHECK constraints in a single file (see §1).

3. How migrations are applied

```
# Apply all pending migrations (the 001..015 chain)
make migrate-up
    # → scripts/run-migrations.sh up

# Roll back exactly one migration
make migrate-down
    # → scripts/run-migrations.sh down 1
```

Details (Makefile:76–84, scripts/run-migrations.sh):

- DATABASE_URL defaults to postgres://helix:helix@localhost:5432/helix_cluster?sslmode=disable (Makefile:76); override it for non-local targets. The script can also assemble the DSN from DB_HOST/DB_PORT/DB_NAME/DB_USER/DB_PASS/DB_SSLMODE (scripts/run-migrations.sh:13–35).
- The script requires the golang-migrate migrate binary on PATH or at ~/go/bin/migrate; otherwise it errors with install instructions (scripts/run-migrations.sh:20–30).
- Additional sub-commands: up [N], down [N], version, force V, create NAME, validate (up → down 15 → up), status (scripts/run-migrations.sh:37–123).

The consolidated 0001_primary_schema.sql is applied separately by the Go helper internal/schema.ApplyPrimarySchema, which pipes the file to psql -f - (works for both local DSN and podman exec container transports) (internal/schema/schema.go:279–308).

4. Trigger functions

Defined in `0001_primary_schema.sql:7-56` (and the equivalent un-prefixed versions in `015_triggers_and_functions.up.sql:7-61`).

helix_update_updated_at_column() / update_updated_at_column()

BEFORE UPDATE row trigger. Sets `NEW.updated_at = NOW()` and returns `NEW`. Attached to every table carrying an `updated_at` column.

helix_audit_trigger() / audit_trigger()

AFTER INSERT OR UPDATE OR DELETE row trigger, SECURITY DEFINER (per code comment, so it can always INSERT into `audit_log`). It inserts a row into `audit_log` with: - `event_type = <TG_OP>_<TG_TABLE_NAME>` (e.g. `INSERT_nodes`) - `severity = WARNING` for DELETE, else `INFO` - `actor = current_setting('app.current_user', TRUE)` or `'system'` - `resource_type = table name`, `resource_id = row id::TEXT` - `action = the operation`, `details = to_jsonb(OLD|NEW)`

Trigger attachments:

Table	updated_at trigger	audit trigger
nodes	yes (helix_nodes_updated_at)	yes (helix_nodes_audit)
gpu_devices	yes	no
sessions	yes	yes (chain 015 only attaches audit to nodes/ sessions/build_jobs/ users)
reservations	yes	no
users	yes	yes
llm_advisories	yes	no
build_jobs	yes	yes
network_policies	yes	no
cluster_config	yes	no

In the migrate chain (015), `updated_at` triggers are attached to `nodes`, `gpu_devices`, `sessions`, `resource_allocations`, `scheduling_queue`, `build_jobs`, `users`; audit triggers to `nodes`, `sessions`, `build_jobs`, `users`.

5. Tables

Column tables below are grounded in `0001_primary_schema.sql`. PK = primary key, FK = foreign key, U = UNIQUE, NN = NOT NULL.

5.1 nodes — cluster node registry

0001_primary_schema.sql:62-115 · 001_create_nodes.up.sql

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT gen_random_uuid()
hostname	VARCHAR(255)	NN
ip_addresses	INET[]	NN
wg_pubkey	TEXT	NN, U (nodes_wg_pubkey_unique)
spiffe_id	TEXT	NN, U (nodes_spiffe_id_unique)
status	VARCHAR(20)	NN, DEFAULT 'JOINING', CHECK ∈ {JOINING, READY, DRAINING, OFFLINE, MAINTENANCE, EVICTED}
role	VARCHAR(20)	NN, DEFAULT 'WORKER', CHECK ∈ {WORKER, CONTROL_PLANE, GPU_WORKER, EDGE, OBSERVER}
cpu_arch	VARCHAR(20)	NN
cpu_cores	INT	NN, CHECK > 0
cpu_threads	INT	NN, CHECK >= cpu_cores
memory_bytes	BIGINT	NN, CHECK > 0
gpu_count	INT	NN, DEFAULT 0, CHECK >= 0
storage_bytes	BIGINT	NN, CHECK > 0
labels	JSONB	NN, DEFAULT '{}'
region	VARCHAR(100)	nullable
version	VARCHAR(50)	NN
joined_at	TIMESTAMPTZ	NN, DEFAULT NOW()
last_seen	TIMESTAMPTZ	NN, DEFAULT NOW()
left_at	TIMESTAMPTZ	nullable
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()
updated_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: idx_nodes_status, idx_nodes_role, idx_nodes_region, idx_nodes_labels (GIN on labels), idx_nodes_last_seen, idx_nodes_hostname. **Triggers:** helix_nodes_updated_at, helix_nodes_audit.

5.2 gpu_devices — per-node GPU inventory

0001_primary_schema.sql:121–152 · 002_create_gpu_devices.up.sql

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT gen_random_uuid()
node_id	UUID	NN, FK → nodes(id) ON DELETE CASCADE
vendor	VARCHAR(20)	NN, CHECK ∈ {NVIDIA, AMD, INTEL, APPLE, QUALCOMM, OTHER}
model	VARCHAR(100)	NN
driver_version	VARCHAR(50)	NN
api	VARCHAR(20)	NN, CHECK ∈ {CUDA, ROCm, Metal, Vulkan, OpenCL, OTHER}
api_version	VARCHAR(20)	NN
total_memory	BIGINT	NN, CHECK > 0
compute_units	INT	NN, CHECK > 0
features	TEXT[]	NN, DEFAULT '{}'
attributes	JSONB	NN, DEFAULT '{}'
status	VARCHAR(20)	NN, DEFAULT 'AVAILABLE', CHECK ∈ {AVAILABLE, IN_USE, RESERVED, OFFLINE, FAULT}
allocated_to	UUID	nullable (no FK declared)
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()
updated_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: idx_gpu_devices_node_id, idx_gpu_devices_status,
idx_gpu_devices_vendor, idx_gpu_devices_model. **Trigger:**
helix_gpu_devices_updated_at.

5.3 sessions — interactive/batch compute sessions

0001_primary_schema.sql:158–200 · 003_create_sessions.up.sql

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT gen_random_uuid()
name	VARCHAR(255)	NN
owner	TEXT	NN

Column	Type	Constraints / Default
status	VARCHAR(20)	NN, DEFAULT 'CREATING', CHECK $\in$ {CREATING, RUNNING, PAUSED, TERMINATING, TERMINATED, FAILED}
mode	VARCHAR(20)	NN, DEFAULT 'INTERACTIVE', CHECK $\in$ {INTERACTIVE, BATCH, DAEMON, NOTEBOOK}
backend	VARCHAR(20)	NN, DEFAULT 'TMUX', CHECK $\in$ {TMUX, WASM, CONTAINER, NATIVE}
backend_id	TEXT	nullable
node_id	UUID	FK $\rightarrow$ nodes(id) , nullable
cpu_request	INT	NN, DEFAULT 1000, CHECK > 0
memory_request	BIGINT	NN, DEFAULT 1073741824, CHECK > 0
gpu_request	JSONB	DEFAULT NULL
priority	INT	NN, DEFAULT 50, CHECK 0..100
labels	JSONB	NN, DEFAULT '{}'
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()
started_at	TIMESTAMPTZ	nullable
terminated_at	TIMESTAMPTZ	nullable
updated_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: idx_sessions_owner, idx_sessions_status, idx_sessions_node_id, idx_sessions_mode, idx_sessions_priority, idx_sessions_labels (GIN). **Trigger:** helix_sessions_updated_at (+ helix_sessions_audit in §1b source / chain).

5.4 session_windows — tmux-style windows within a session

0001_primary_schema.sql:206–221 · 004_create_session_windows.up.sql

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT gen_random_uuid()

Column	Type	Constraints / Default
session_id	UUID	NN, FK → sessions(id) ON DELETE CASCADE
name	VARCHAR(255)	NN
layout	VARCHAR(50)	NN, DEFAULT 'tiled', CHECK ∈ {tiled, even-horizontal, even-vertical, main-horizontal, main-vertical}
active	BOOLEAN	NN, DEFAULT FALSE
crdt_state	JSONB	DEFAULT NULL
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: idx_session_windows_session_id, idx_session_windows_active.
No updated_at column / trigger.

5.5 session_panes — panes within a window (placed on a node/GPU)

0001_primary_schema.sql:227–251 · 005_create_session_panes.up.sql

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT gen_random_uuid()
window_id	UUID	NN, FK → session_windows(id) ON DELETE CASCADE
node_id	UUID	FK → nodes(id) , nullable
command	TEXT	nullable
working_dir	TEXT	nullable
environment	JSONB	NN, DEFAULT '{}'
cpu_limit	INT	nullable, CHECK IS NULL OR > 0
memory_limit	BIGINT	nullable, CHECK IS NULL OR > 0
gpu_id	UUID	FK → gpu_devices(id) , nullable
status	VARCHAR(20)	NN, DEFAULT 'CREATING', CHECK ∈ {CREATING, RUNNING, STOPPED, FAILED, MIGRATING}
crdt_state	JSONB	DEFAULT NULL

Column	Type	Constraints / Default
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: idx_session_panes_window_id, idx_session_panes_node_id, idx_session_panes_gpu_id, idx_session_panes_status.

5.6 reservations — resource reservations

0001_primary_schema.sql:257–284 (migrate chain: table named **resource_allocations**, 006_*, no CHECK constraints)

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT gen_random_uuid()
session_id	UUID	NN, FK → sessions(id) ON DELETE CASCADE
node_id	UUID	NN, FK → nodes(id)
cpu_millicores	INT	NN, CHECK > 0
memory_bytes	BIGINT	NN, CHECK > 0
gpu_ids	UUID[]	NN, DEFAULT '{}'
status	VARCHAR(20)	NN, DEFAULT 'PENDING', CHECK ∈ {PENDING, ACTIVE, RELEASED, EXPIRED, FAILED}
expires_at	TIMESTAMPTZ	NN, CHECK > created_at
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()
updated_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: idx_reservations_session_id, idx_reservations_node_id, idx_reservations_status, idx_reservations_expires_at. **Trigger:** helix_reservations_updated_at. > In the migrate chain (006), resource_allocations.session_id FK has **no** CASCADE, > gpu_ids is nullable (no DEFAULT '{}'), and there are no CHECK constraints.

5.7 scheduling_queue — pending scheduling requests (*migrate chain only*)

007_create_scheduling_queue.up.sql · **not present in** 0001_primary_schema.sql

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT gen_random_uuid()
request	JSONB	NN
priority	INT	NN, DEFAULT 50

Column	Type	Constraints / Default
status	VARCHAR(20)	NN, DEFAULT 'PENDING'
scheduled_at	TIMESTAMPTZ	nullable
started_at	TIMESTAMPTZ	nullable
completed_at	TIMESTAMPTZ	nullable
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()
updated_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: idx_scheduling_status, idx_scheduling_priority, idx_scheduling_created. **Trigger (chain 015):** scheduling_queue_updated_at.

5.8 health_snapshots — periodic per-node health scores

0001_primary_schema.sql:418–445 (migrate chain: table named **health_scores**, 008_*, no CHECK ranges)

Column	Type	Constraints / Default
id	BIGSERIAL	PK
node_id	UUID	NN, FK → nodes(id) ON DELETE CASCADE
overall_score	INT	NN, CHECK 0..100
cpu_score	INT	NN, CHECK 0..100
memory_score	INT	NN, CHECK 0..100
disk_score	INT	NN, CHECK 0..100
network_score	INT	NN, CHECK 0..100
gpu_score	INT	NN, CHECK 0..100
temperature_score	INT	NN, CHECK 0..100
services_score	INT	NN, CHECK 0..100
predictions	JSONB	NN, DEFAULT '[]'
metrics	JSONB	NN, DEFAULT '{}'
recorded_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: idx_health_snapshots_node_id, idx_health_snapshots_recorded_at, idx_health_snapshots_overall. > Migrate chain (008 health_scores): node_id FK has no CASCADE, no 0..100 CHECKs.

5.9 llm_advisories — LLM “brain” advisories

0001_primary_schema.sql:451–487 (migrate chain: table named **advisories**, 009_*, no CHECK, no updated_at)

Column	Type	Constraints / Default
id	UUID	

Column	Type	Constraints / Default
type	VARCHAR(30)	PK , DEFAULT gen_random_uuid() NN, CHECK ∈ {SCALE_UP, SCALE_DOWN, MIGRATE, REBALANCE, DRAIN, EVICT, ALERT, OPTIMIZE, SCHEDULE}
description	TEXT	NN
rationale	TEXT	NN
proposed_action	JSONB	NN
confidence	FLOAT	NN, CHECK 0.0..1.0
risk_level	VARCHAR(10)	NN, CHECK ∈ {LOW, MEDIUM, HIGH, CRITICAL}
auto_approve	BOOLEAN	NN, DEFAULT FALSE
status	VARCHAR(20)	NN, DEFAULT 'PENDING', CHECK ∈ {PENDING, APPROVED, REJECTED, APPLIED, FAILED, EXPIRED}
applied_by	TEXT	nullable
applied_at	TIMESTAMPTZ	nullable
result	JSONB	nullable
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()
updated_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: idx_llm_advisories_status, idx_llm_advisories_type, idx_llm_advisories_risk_level, idx_llm_advisories_created_at.

Trigger: helix_llm_advisories_updated_at. > Migrate chain (009 advisories): type is VARCHAR(20), there is **no** updated_at > column, and no CHECK constraints.

5.10 audit_log — append-only audit trail (partitioned by month)

0001_primary_schema.sql:318–376 · 010_create_audit_log.up.sql

Column	Type	Constraints / Default
id	BIGSERIAL	part of PK
timestamp	TIMESTAMPTZ	NN, DEFAULT NOW(), partition key
event_type	VARCHAR(50)	NN
severity	VARCHAR(10)	

Column	Type	Constraints / Default
		NN, DEFAULT 'INFO', CHECK ∈ {DEBUG, INFO, WARNING, ERROR, CRITICAL}
actor	TEXT	NN
resource_type	VARCHAR(50)	NN
resource_id	TEXT	nullable
action	VARCHAR(50)	NN
details	JSONB	NN, DEFAULT '{}'
source_ip	INET	nullable
session_id	UUID	nullable

Primary key: composite (id, timestamp) (required because the table is PARTITION BY RANGE (timestamp)). **Partitions (0001):** audit_log_default (DEFAULT) + monthly audit_log_2026_06 ... audit_log_2027_06. **Indexes:** idx_audit_log_timestamp, idx_audit_log_event_type, idx_audit_log_actor, idx_audit_log_resource (resource_type, resource_id), idx_audit_log_severity, idx_audit_log_session_id. > Migrate chain (010): partitions only 2026-06...2026-12, **no** DEFAULT partition, > **no** severity CHECK, 4 indexes (no severity/session_id indexes).

5.11 users — shadow of the OIDC/SPIFFE identity provider

0001_primary_schema.sql:382–412 · 013_create_users.up.sql

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT gen_random_uuid()
spiffe_id	TEXT	NN, U (users_spiffe_id_unique)
email	TEXT	nullable
name	VARCHAR(255)	nullable
role	VARCHAR(20)	NN, DEFAULT 'USER', CHECK ∈ {SUPERADMIN, ADMIN, OPERATOR, USER, READONLY, SERVICE}
quota_cpu	INT	NN, DEFAULT 8000, CHECK >= 0
quota_memory	BIGINT	NN, DEFAULT 17179869184, CHECK >= 0
quota_gpu	INT	NN, DEFAULT 0, CHECK >= 0
labels	JSONB	NN, DEFAULT '{}'
last_login	TIMESTAMPTZ	nullable

Column	Type	Constraints / Default
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()
updated_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: idx_users_spiffe_id, idx_users_role, idx_users_email, idx_users_labels (GIN). **Triggers:** helix_users_updated_at (+ audit in \$1b source / chain). > Migrate chain (013): only idx_users_spiffe index; no role/email/labels indexes, > no CHECK constraints.

5.12 migration_history — session live-migration ledger

0001_primary_schema.sql:290–312 ·
014_create_migration_history.up.sql

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT gen_random_uuid()
session_id	UUID	NN, FK → sessions(id)
source_node	UUID	NN, FK → nodes(id)
target_node	UUID	NN, FK → nodes(id)
method	VARCHAR(20)	NN, CHECK ∈ {CHECKPOINT, LIVE, COLD, SNAPSHOT}
duration_ms	INT	NN, CHECK >= 0
data_size_bytes	BIGINT	nullable
success	BOOLEAN	NN
error_message	TEXT	nullable
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Additional CHECK: source_node <> target_node (mh_nodes_differ).
Indexes (0001): idx_migration_history_session_id, idx_migration_history_source_node, idx_migration_history_target_node, idx_migration_history_success. > Migrate chain (014): no CHECK constraints; 3 indexes (no _success index).

5.13 build_jobs — build subsystem jobs

0001_primary_schema.sql:493–528 · 011_create_build_jobs.up.sql

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT gen_random_uuid()
name	VARCHAR(255)	NN
status	VARCHAR(20)	NN, DEFAULT 'PENDING', CHECK ∈ {PENDING, QUEUED,

Column	Type	Constraints / Default
		RUNNING, SUCCEEDED, FAILED, CANCELLED}
mode	VARCHAR(20)	NN, DEFAULT 'BATCH', CHECK $\in$ {BATCH, STREAMING, INCREMENTAL, PARALLEL}
cpu_request	INT	NN, DEFAULT 1000, CHECK > 0
memory_request	BIGINT	NN, DEFAULT 1073741824, CHECK > 0
gpu_request	JSONB	DEFAULT NULL
node_id	UUID	FK $\rightarrow$ nodes(id) , nullable
priority	INT	NN, DEFAULT 50, CHECK 0..100
labels	JSONB	NN, DEFAULT '{}'
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()
started_at	TIMESTAMPTZ	nullable
completed_at	TIMESTAMPTZ	nullable
updated_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: idx_build_jobs_status, idx_build_jobs_node_id, idx_build_jobs_mode, idx_build_jobs_created_at, idx_build_jobs_priority. **Triggers:** helix_build_jobs_updated_at (+ audit).

5.14 build_artifacts — outputs of build jobs

0001_primary_schema.sql:534-553 · 012_create_build_artifacts.up.sql

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT gen_random_uuid()
job_id	UUID	NN, FK $\rightarrow$ build_jobs(id) ON DELETE CASCADE
artifact_hash	TEXT	NN
size_bytes	BIGINT	NN, CHECK > 0
storage_path	TEXT	NN
artifact_type	VARCHAR(30)	NN, DEFAULT 'BINARY', CHECK $\in$ {BINARY, CONTAINER_IMAGE,

Column	Type	Constraints / Default
		WASM, LIBRARY, CONFIG, REPORT}
metadata	JSONB	NN, DEFAULT '{}'
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: idx_build_artifacts_job_id, idx_build_artifacts_hash, idx_build_artifacts_created_at, idx_build_artifacts_type. > Migrate chain (012): no artifact_type column, no CHECK; 3 indexes (no _type).

5.15 network_policies — firewall/network policy rules (0001 only)

0001_primary_schema.sql:559-606 · not present in migrate chain

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT gen_random_uuid()
name	VARCHAR(255)	NN, U (np_name_unique)
direction	VARCHAR(10)	NN, CHECK ∈ {INGRESS, EGRESS, BOTH}
action	VARCHAR(10)	NN, CHECK ∈ {ALLOW, DENY, LOG, RATE_LIMIT}
protocol	VARCHAR(10)	nullable, CHECK IS NULL OR ∈ {TCP, UDP, ICMP, ESP, AH, ANY}
src_cidr	CIDR	nullable
dst_cidr	CIDR	nullable
src_port_min	INT	nullable, CHECK IS NULL OR 0..65535
src_port_max	INT	nullable
dst_port_min	INT	nullable, CHECK IS NULL OR 0..65535
dst_port_max	INT	nullable
priority	INT	NN, DEFAULT 100, CHECK 0..65535
enabled	BOOLEAN	NN, DEFAULT TRUE
labels	JSONB	NN, DEFAULT '{}'
description	TEXT	nullable
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()
updated_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Additional CHECKs: `src_port_min <= src_port_max (np_src_port_order)`, `dst_port_min <= dst_port_max (np_dst_port_order)`. **Indexes:** `idx_network_policies_direction`, `idx_network_policies_action`, `idx_network_policies_enabled`, `idx_network_policies_priority`. **Trigger:** `helix_network_policies_updated_at`.

5.16 cluster_config — key/value cluster configuration (0001 only)

0001_primary_schema.sql:612–649 · not present in migrate chain

Column	Type	Constraints / Default
id	UUID	PK , DEFAULT <code>gen_random_uuid()</code>
key	VARCHAR(255)	NN, U (cc_key_unique)
value	JSONB	NN
description	TEXT	nullable
category	VARCHAR(50)	NN, DEFAULT 'general', CHECK $\in \{\text{general, scheduler, network, security, storage, monitoring, ilm, build}\}$
readonly	BOOLEAN	NN, DEFAULT FALSE
version	INT	NN, DEFAULT 1, CHECK > 0
updated_by	TEXT	NN, DEFAULT 'system'
created_at	TIMESTAMPTZ	NN, DEFAULT NOW()
updated_at	TIMESTAMPTZ	NN, DEFAULT NOW()

Indexes: `idx_cluster_config_key`, `idx_cluster_config_category`. **Trigger:** `helix_cluster_config_updated_at`. **Seed rows** (INSERT ... ON CONFLICT (key) DO NOTHING, 0001:643–649): `scheduler.max_queue_depth=1000`, `scheduler.default_priority=50`, `network.mtu=1420`, `security.session_ttl_hours=8`, `monitoring.health_interval_s=30`.

6. Entity-Relationship diagram

Foreign-key relationships from 0001_primary_schema.sql. `network_policies`, `cluster_config`, and `audit_log` carry no FK references (`audit_log.session_id` and `gpu_devices.allocated_to` are plain UUIDs with no declared FK).

```
erDiagram
    nodes ||--o{ gpu_devices : "node_id (CASCADE)"
    nodes ||--o{ sessions : "node_id"
    nodes ||--o{ session_panes : "node_id"
    nodes ||--o{ reservations : "node_id"
```

```

nodes ||--o{ health_snapshots : "node_id (CASCADE)"
nodes ||--o{ build_jobs : "node_id"
nodes ||--o{ migration_history : "source_node"
nodes ||--o{ migration_history : "target_node"

sessions ||--o{ session_windows : "session_id (CASCADE)"
sessions ||--o{ reservations : "session_id (CASCADE)"
sessions ||--o{ migration_history : "session_id"

session_windows ||--o{ session_panes : "window_id (CASCADE)"
gpu_devices ||--o{ session_panes : "gpu_id"

build_jobs ||--o{ build_artifacts : "job_id (CASCADE)"

nodes {
    uuid id PK
    text wg_pubkey UK
    text spiffe_id UK
    varchar status
    varchar role
}
gpu_devices {
    uuid id PK
    uuid node_id FK
    varchar vendor
    varchar status
}
sessions {
    uuid id PK
    uuid node_id FK
    varchar status
    varchar mode
    varchar backend
}
session_windows {
    uuid id PK
    uuid session_id FK
    varchar layout
}
session_panes {
    uuid id PK
    uuid window_id FK
    uuid node_id FK
    uuid gpu_id FK
    varchar status
}
reservations {
    uuid id PK
    uuid session_id FK
    uuid node_id FK
    varchar status
}
health_snapshots {
    bigserial id PK

```

```

        uuid node_id FK
        int overall_score
    }
migration_history {
    uuid id PK
    uuid session_id FK
    uuid source_node FK
    uuid target_node FK
    boolean success
}
build_jobs {
    uuid id PK
    uuid node_id FK
    varchar status
}
build_artifacts {
    uuid id PK
    uuid job_id FK
    text artifact_hash
}
users {
    uuid id PK
    text spiffe_id UK
    varchar role
}
llm_advisories {
    uuid id PK
    varchar type
    varchar status
}
scheduling_queue {
    uuid id PK
    jsonb request
    varchar status
}
network_policies {
    uuid id PK
    varchar name UK
    varchar direction
    varchar action
}
cluster_config {
    uuid id PK
    varchar key UK
    varchar category
}
audit_log {
    bigserial id PK
    timestampz timestamp PK
    varchar event_type
    varchar severity
}

```

Standalone tables (no FK edges): `users`, `llm_advisories / advisories`, `scheduling_queue` (chain only), `network_policies` (0001 only), `cluster_config` (0001 only), `audit_log`.

7. Source files

- `migrations/postgresql/0001_primary_schema.sql` — consolidated authoritative schema (15 tables, CHECK constraints, ~53 indexes, triggers, seed config). Applied via `internal/schema.ApplyPrimarySchema (psql -f -)`.
- `migrations/postgresql/001_*.up.sql ... 015_*.up.sql` (+ paired `*.down.sql`) — the `golang-migrate` chain applied by `make migrate-up`.
- `scripts/run-migrations.sh` — the `golang-migrate` runner.
- `Makefile (migrate-up / migrate-down)` — operator entry points.
- `internal/schema/schema.go` — Go loader/applier for `0001_primary_schema.sql`.